MODULE 3: DATA VISUALIZATION (09 PERIODS)

Course Code: 22DS102006 | **B.Tech Computer Science & Engineering (Data Science)**

Detailed 10-Page Comprehensive Study Notes: Matplotlib API, Pandas/Seaborn Integration, Distribution Analysis, Categorical Plots & Interactive Visualization Tools.

1. Introduction to Data Visualization

Data visualization is the graphic representation of information and data. By converting structured and unstructured datasets into visual signals like charts, graphs, and maps, visualization tools provide an accessible way to discover trends, outliers, and patterns in data.

In data science pipelines, visualization serves two distinct primary goals: Exploratory Data Analysis (EDA) for hypothesis generation, and Explanatory Analytics for conveying complex findings to stakeholders.

Core Purpose of Visual Data Analysis

- **Cognitive Acceleration:** Human brain processes visual marks orders of magnitude faster than numerical matrices.
- **Anomaly Identification:** Spotting structural gaps, multi-modal clusters, and extreme outliers immediately.
- **Model Diagnostics:** Visualizing residual distribution, overfitting curves, and feature influence maps.

Anscombe's Quartet & Data Inspection

Anscombe's Quartet demonstrates four datasets with identical statistical properties (mean, variance, correlation, and regression line) that display radically different distributions when plotted visually, underscoring why tabular summaries alone are insufficient.

2. Matplotlib Architecture & Object-Oriented API

Matplotlib is structured in a three-layer hierarchy: the Backend Layer (rendering targets), the Artist Layer (figure primitives), and the Scripting/Object-Oriented Layer (user APIs).

The Object-Oriented Hierarchy

While the state-based `pyplot` interface provides quick plotting, the Object-Oriented (OO) API provides explicit, deterministic control over figure structures.

- **Figure (`matplotlib.figure.Figure`):** The top-level canvas container holding all canvas components, subplots, legends, and colorbars.
- **Axes (`matplotlib.axes.Axes`):** The actual plotting region/bounding box containing coordinate systems, tick marks, labels, and data artists.
- **Axis (`matplotlib.axis.Axis`):** Individual numerical/categorical number lines managing tick mark positioning and label formatting.

> **Pyplot vs. Object-Oriented API Syntax**
>
> **Pyplot (Stateful):** `plt.plot(x, y); plt.title("Title"); plt.show()`
>
> **OO API (Explicit):** `fig, ax = plt.subplots(); ax.plot(x, y); ax.set_title("Title")`

3. Subplot Layouts, Grids & Customization

Multi-panel figures allow comparative analysis across sub-populations. Matplotlib provides flexible grid specification mechanisms using `plt.subplots` and `GridSpec`.

Subplot Grid Creation

Subplots can be arranged in uniform grid matrices or custom spanning structures using relative axis positioning.

```
      import matplotlib.pyplot as plt import numpy as np # 1. Standard 2x2 Subplot Matrix
fig, axes = plt.subplots(2, 2, figsize=(10, 6), sharex=True) axes[0, 0].plot([1, 2, 3], [4,
5, 6], color='navy') axes[0, 0].set_title("Subplot (0,0)") # 2. Advanced Custom Grid via
GridSpec fig2 = plt.figure(figsize=(10, 6)) gs = fig2.add_gridspec(3, 3) ax_main =
fig2.add_subplot(gs[0:2, :]) # Spans top 2 rows ax_sub1 = fig2.add_subplot(gs[2, 0:2]) #
Bottom left ax_sub2 = fig2.add_subplot(gs[2, 2]) # Bottom right ax_main.set_title("Main Span
Area") plt.tight_layout()
```

Best Practices for Multi-Plot Layouts

Always utilize `sharex=True` or `sharey=True` when comparing axes across panels to maintain consistent scale interpretation. Use `plt.tight_layout()` or `constrained_layout=True` to prevent label clipping.

4. Built-in Dataframe Plotting in Pandas

Pandas integrates Matplotlib functionality directly onto `DataFrame` and `Series` objects via the `.plot()` accessor method, enabling rapid visualization of structured tabular datasets.

Pandas Plotting Methods

- **Line Plot (`df.plot.line()`):** Default visualization for time-series and ordered index series.
- **Bar Charts (`df.plot.bar()` / `df.plot.barh()`):** Plots categorical values against numeric metrics with automatic grouped/stacked column handling.
- **Histograms & KDE (`df.plot.hist()` / `df.plot.kde()`):** Shows continuous distribution density curves across DataFrame columns.
- **Scatter & Hexbin (`df.plot.scatter()` / `df.plot.hexbin()`):** Evaluates two-variable relationships; hexbin handles dense overplotted coordinate points.

```
import pandas as pd import numpy as np # Sample Dataframe creation df = pd.DataFrame({ 'Quarter': ['Q1', 'Q2', 'Q3', 'Q4'], 'Product_A': [150, 200, 250, 300], 'Product_B': [90, 180, 210, 260] }) # Stacked Bar Plot directly from DataFrame ax = df.plot.bar(x='Quarter', stacked=True, figsize=(8, 4), colormap='viridis') ax.set_ylabel("Revenue ($k)") ax.set_title("Quarterly Product Revenue")
```

5. Seaborn: Statistical Data Visualization

Seaborn is a high-level statistical plotting library built on top of Matplotlib that integrates tightly with Pandas DataFrames. It automatically handles statistical aggregation, confidence interval computation, and color palette assignment.

Key Features of Seaborn

- **DataFrame Awareness:** Accepts long-form or wide-form DataFrames directly using string column mapping (`x="col1", y="col2", hue="class"`).
- **Built-in Statistical Estimation:** Calculates sample means, medians, bootstrapping confidence intervals (95% CI), and kernel density estimates natively during render.
- **Aesthetic Themes:** Simple style configurations via `sns.set_theme(style="whitegrid")` and palette mapping.

> **Seaborn Plot Categories**
>
> **Relational:** `sns.scatterplot()`, `sns.lineplot()`, `sns.relplot()`
>
> **Categorical:** `sns.barplot()`, `sns.boxplot()`, `sns.violinplot()`, `sns.catplot()`
>
> **Distribution:** `sns.histplot()`, `sns.kdeplot()`, `sns.ecdfplot()`, `sns.displot()`

6. Continuous Distribution & Relational Plots

Understanding continuous distributions is vital for verifying statistical assumptions (e.g., normality) and detecting data skewness.

Histograms, KDE & Box Plots

- **Histogram & KDE (`sns.histplot`):** Quantifies frequency across continuous bins combined with Gaussian kernel smoothing density overlays.
- **Box Plots (`sns.boxplot`):** Visualizes 5-number summaries (Min, Q1, Median, Q3, Max) and identifies outliers via Interquartile Range (`1.5 × IQR` rule).
- **Violin Plots (`sns.violinplot`):** Combines a box plot with a rotated kernel density plot on each side to expose multi-modal distribution structures.

```
     import seaborn as sns import matplotlib.pyplot as plt # Loading sample dataset df =
sns.load_dataset("tips") # 1. Joint Plot (Scatter + Distribution Margins) g =
sns.jointplot(data=df, x="total_bill", y="tip", hue="time", kind="scatter") # 2. Violin Plot
with Split Hue plt.figure(figsize=(8, 4)) sns.violinplot(data=df, x="day", y="total_bill",
hue="smoker", split=True, palette="muted") plt.title("Bill Distribution by Day & Smoking
Status")
```

7. Categorical Analysis & Multi-Plot Facet Grids

When dataset dimension counts exceed 3 or 4 features, Multi-plot grids break complex interactions into small multiples conditioned on categorical variables.

Faceting via FacetGrid & PairPlot

- **FacetGrid (`sns.FacetGrid`):** Maps a dataset onto a grid of panels defined by row and column categorical factors.
- **PairPlot (`sns.pairplot`):** Draws a grid of pairwise scatter plots for every continuous numerical column, with histograms along the diagonal.
- **Heatmaps (`sns.heatmap`):** Renders 2D matrices (such as correlation matrices or confusion matrices) with color-encoded magnitudes.

```
      import seaborn as sns import matplotlib.pyplot as plt tips = sns.load_dataset("tips") # 1. FacetGrid across Row/Column Categories g = sns.FacetGrid(tips, col="time", row="smoker", margin_titles=True) g.map_dataframe(sns.scatterplot, x="total_bill", y="tip", hue="sex") g.add_legend() # 2. Correlation Matrix Heatmap plt.figure(figsize=(6, 4)) corr = tips.corr(numeric_only=True) sns.heatmap(corr, annot=True, cmap="coolwarm", vmin=-1, vmax=1)
```

8. Aesthetics, Color Theory & Plot Customization

Effective visual communication requires thoughtful selection of color spaces, typography, line weights, and structural annotations to avoid visual clutter.

Color Palette Selection Rules

- **Qualitative Palettes (`'Set2'`, `'tab10'`):** Used for unordered categorical variables where color differences denote distinct categories without implying rank.
- **Sequential Palettes (`'Blues'`, `'viridis'`):** Used for ordered continuous numeric data where lighter/darker shades represent magnitude.
- **Diverging Palettes (`'Coolwarm'`, `'Spectral'`):** Used for data with a meaningful midpoint (e.g., zero value, mean temperature anomalies).

> ### Publication-Ready Styling Checklist
>
> 1. Ensure text contrast complies with accessibility standards; avoid pure red/green maps for colorblind readability (use Viridis/Plasma).
> 2. Remove unnecessary chart junk (e.g., top and right axis spines) using `sns.despine()`.
> 3. Always label axes explicitly with measurement units and add explanatory titles.

9. Overview of Other Python Visualization Tools

Beyond Matplotlib and Seaborn, the modern Python ecosystem offers specialized tools for interactive web analytics, geospatial rendering, and large-scale big data visualization.

Interactive & Big Data Libraries

Library	Primary Paradigm	Key Use Case
Plotly	Declarative JSON / Interactive HTML	Web dashboards, hover tooltips, 3D surface charts
Bokeh	Web-ready reactive canvas	Streaming data feeds, interactive browser applications
Altair	Grammar of Graphics (Vega-Lite)	Declarative, statistical data exploration
Datashader	Rasterizing server-side graphics	Visualizing billions of data points without downsampling
Folium / GeoPandas	Leaflet JS integration	Interactive geospatial maps and choropleths

When to Choose Interactive vs. Static

Static graphics (Matplotlib/Seaborn) are preferred for academic papers, PDF reports, and printed publications. Interactive graphics (Plotly/Bokeh) excel in web applications where users need zoom, pan, and filter capabilities.

10. Comprehensive Implementation & Review

Below is a complete end-to-end Python script assembling a multi-panel exploratory dashboard using Seaborn and Matplotlib Object-Oriented APIs.

```
      import matplotlib.pyplot as plt import seaborn as sns # Set global theme and context
sns.set_theme(style="ticks", palette="muted") fig, axes = plt.subplots(1, 2, figsize=(11,
4.5)) # Load dataset df = sns.load_dataset("diamonds").query("carat < 2.5") # Panel 1:
Relational Scatter with Alpha sns.scatterplot(data=df, x="carat", y="price", hue="cut",
alpha=0.5, ax=axes[0]) axes[0].set_title("Diamond Price vs. Carat Weight") # Panel 2:
Categorical Boxplot sns.boxplot(data=df, x="color", y="price", palette="Blues", ax=axes[1])
axes[1].set_title("Price Distribution across Color Grades") sns.despine() plt.tight_layout()
plt.savefig("diamond_analysis.png", dpi=300)
```

Module 3 Exam Review Questions

- Explain the structural difference between Matplotlib's `pyplot` state-machine and its Object-Oriented API.
- Compare Box Plots and Violin Plots in terms of information density and identifying multimodal distributions.
- Describe how Seaborn's `FacetGrid` manages multi-plot layouts based on categorical factors.